

Reporte Técnico de Actividades Práctico-Experimentales Nro. 00X

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Emerson Chamba Galarza
Asignatura	Teoría de la programación
Ciclo	1 A
Unidad	3
Resultado de aprendizaje de la unidad	Desarrolla aplicaciones utilizando el principio de la programación modular y estructuras de datos simples y/o estáticas compuestas, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Tipo	Individual o Grupal
Título de la Práctica	Construcción de funciones y procedimientos en un lenguaje de programación
Nombre del Docente	Lissette Geoconda López Faicán
Fecha	Jueves 8 de enero del 2026 Jueves 15 de enero del 2026
Horario	10h30 – 13h30
Lugar	Aula física asignada al paralelo.
Tiempo planificado en el Sílabo	6 horas

2. Objetivo(s) de la Práctica

- Aplicar los fundamentos de la programación modular mediante la construcción y uso de funciones y procedimientos, para resolver un problema real, garantizando un código estructurado, reutilizable y correctamente documentado.

3. Materiales, Reactivos, Equipos y Herramientas

- IDE de programación: Visual Studio Code u otro entorno compatible.
- Lenguaje de programación: C (según los contenidos de la unidad).
- Computador personal con sistema operativo Windows, Linux o macOS.
- Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).
- Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF.
- Conexión a internet estable para acceder a recursos digitales y software en línea.
- Aula física asignada al paralelo.

4. Procedimiento / Metodología Ejecutada

Se aplicó la metodología de **aprendizaje basado en problemas**, siguiendo estas etapas:

- **Análisis y Contextualización:** Se identificó la necesidad de calcular la nota final mediante componentes evaluativos (ACD, APE, AA, ES) con sus respectivos pesos y escalas cualitativas.
- **Codificación Modular:** Se trasladó la solución al lenguaje C, creando funciones específicas para cada tipo de aprendizaje para asegurar la reutilización del código.
- **Implementación de Reglas de Negocio:** Se programaron las validaciones para que las notas ingresadas estén en el rango de **0.0 a 10.0**.
- **Pruebas y Depuración:** Se compiló el programa en el IDE para verificar la correcta obtención de los promedios y la clasificación final (Aprobado, Supletorio o Reprobado)

5. Resultados

Contextualización y descripción del problema

El problema consiste en automatizar el cálculo de la nota final de un estudiante en la asignatura de Teoría de la Programación. Se requiere un sistema que procese cuatro componentes evaluativos por cada unidad: Aprendizaje en Contacto con el Docente (ACD), Aprendizaje Práctico Experimental (APE), Aprendizaje Autónomo (AA) y una Evaluación Sumativa (ES). El programa debe calcular los promedios ponderados de cada sección, obtener la nota final del ciclo tras tres unidades y asignar una clasificación cualitativa (Aprobado, Supletorio o Reprobado) basada en una escala de 0.0 a 10.0

Código fuente del programa

```
#include <stdio.h>
```

```
// DECLARACION DE FUNCIONES
```

```
float calcularSeccion(const char* nombre, float peso);
```

```
float calcularES();
```

```
const char* obtenerClasificacion(float nota);
```

```
int main() {
```

```
    int NUMEROUNIDADES = 3;
```

```
    float promedioTotal = 0;
```

```
    for (int i = 1; i <= NUMEROUNIDADES; i++) {  
        printf("\nUNIDAD %i\n", i);
```

```
        float notaACD = calcularSeccion("ACD", 0.20);
```

```
        float notaAPE = calcularSeccion("APE", 0.25);
```

```
        float notaAA = calcularSeccion("AA", 0.20);
```

```
        float notaES = calcularES();
```

```
        float promedioUnidad = notaACD + notaAPE + notaAA + notaES;
```

```
        printf("\n-> Promedio Unidad %i: %.2f\n", i, promedioUnidad);
```

```
        promedioTotal += promedioUnidad;
```

```
    }
```

```
    float notaFinalCiclo = promedioTotal / NUMEROUNIDADES;
```

```
    // Clasificación SOLO con la nota final del ciclo
```



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

```
printf("\nPROMEDIO FINAL DEL CICLO: %.2f", notaFinalCiclo);
printf("\nCLASIFICACION FINAL: %s", obtenerClasificacion(notaFinalCiclo));

return 0;
}

// FUNCIONES

float calcularSeccion(const char* nombre, float peso) {
    int numeroActividades;
    float notaActividad;
    float notaAcumulativa = 0;

    printf("\nIngrese el numero de actividades para %s: ", nombre);
    scanf("%i", &numeroActividades);

    for (int i = 1; i <= numeroActividades; i++) {
        do {
            printf(" Nota %i: ", i);
            scanf("%f", &notaActividad);
            if (notaActividad < 0 || notaActividad > 10)
                printf(" Rango permitido: 0 - 10\n");
        } while (notaActividad < 0 || notaActividad > 10);

        notaAcumulativa += notaActividad;
    }

    return (notaAcumulativa / numeroActividades) * peso;
}

float calcularES() {
    float notaPortafolio, notaExamen;

    printf("\nComponente ES (Evaluacion Sumativa):\n");

    do {
        printf("Nota portafolio: ");
        scanf("%f", &notaPortafolio);
    } while (notaPortafolio < 0 || notaPortafolio > 10);

    do {
        printf("Nota examen: ");
        scanf("%f", &notaExamen);
    } while (notaExamen < 0 || notaExamen > 10);

    return ((notaPortafolio * 0.40) + (notaExamen * 0.60)) * 0.35;
}

const char* obtenerClasificacion(float nota) {
    if (nota >= 9.0)
        return "Excelente";
    else if (nota >= 7.0)
        return "Bueno";
    else if (nota >= 5.0)
        return "Regular";
    else
        return "Deficiente";
}
```

Pruebas: ejecución de un caso de prueba

Se ha realizado una prueba de ejecución considerando que el estudiante obtiene una nota de 9.0 en todas sus actividades, portafolio y examen en las tres unidades del ciclo.

Datos de entrada para cada unidad:

- ACD: 2 actividades con nota de 9.0 cada una.



- APE: 2 actividades con nota de 9.0 cada una.
- AA: 2 actividades con nota de 9.0 cada una.
- ES: Portafolio (9.0) y Examen (9.0).

Desglose de cálculos por Unidad:

ACD: Promedio (9.0) $\times$ 0.20 = 1.80 pts.

APE: Promedio (9.0) $\times$ 0.25 = 2.25 pts.

AA: Promedio (9.0) $\times$ 0.20 = 1.80 pts.

ES: $((9.0 \times 0.40) + (9.0 \times 0.60)) \times 0.35 = 3.15$ pts

Total Unidad: 1.80 + 2.25 + 1.80 + 3.15 = 9.00 pts.

Ejecución en Consola

```
PS C:\Users\Usuario\Documents\Programacion_vsc\c> .\main.exe

UNIDAD 1

Ingrese el numero de actividades para ACD: 2
Nota 1: 9
Nota 2: 9

Ingrese el numero de actividades para APE: 2
Nota 1: 9
Nota 2: 9

Ingrese el numero de actividades para AA: 2
Nota 1: 9
Nota 2: 9

Componente ES (Evaluacion Sumativa):
Nota portafolio: 9
Nota examen: 9

-> Promedio Unidad 1: 9.00
```

```
UNIDAD 2

Ingrese el numero de actividades para ACD: 2
Nota 1: 9
Nota 2: 9

Ingrese el numero de actividades para APE: 2
Nota 1: 9
Nota 2: 9

Ingrese el numero de actividades para AA: 2
Nota 1: 9
Nota 2: 9

Componente ES (Evaluacion Sumativa):
Nota portafolio: 9
Nota examen: 9

-> Promedio Unidad 2: 9.00
```

```
UNIDAD 3

Ingrese el numero de actividades para ACD: 2
Nota 1: 9
Nota 2: 9

Ingrese el numero de actividades para APE: 2
Nota 1: 9
Nota 2: 9

Ingrese el numero de actividades para AA: 2
Nota 1: 9
Nota 2: 9

Componente ES (Evaluacion Sumativa):
Nota portafolio: 9
Nota examen: 9

-> Promedio Unidad 3: 9.00
```

```
PROMEDIO FINAL DEL CICLO: 9.00
CLASIFICACION FINAL: Excelente
PS C:\Users\Usuario\Documents\Programacion_vsc\c> |
```

6. Preguntas de Control

- **¿Cuál es la diferencia entre una función y un procedimiento?**

La diferencia principal es que una función realiza un cálculo y devuelve un valor específico (como el promedio ponderado de una unidad) , mientras que un procedimiento se encarga de ejecutar una acción sin necesariamente devolver un dato, como por ejemplo imprimir los resultados finales en pantalla.

- **¿Qué ventajas aporta dividir un programa en funciones (modularidad)?**

Dividir el programa ayuda mucho porque:

- El código es más ordenado y fácil de entender.
- Permite reutilizar código, evitando escribir lo mismo varias veces para cada componente evaluativo.
- Es más sencillo encontrar y corregir errores, ya que cada parte (ACD, APE, AA, ES) funciona de manera independiente.
- **¿Qué se mejoraría del programa si se tuviera que usarlo para varios estudiantes?**

Si quisiéramos usarlo para todo un curso, se podrían hacer estas mejoras:

Usar un ciclo (bucle) para que el programa no se cierre y pida los datos de un nuevo estudiante automáticamente.

Utilizar arreglos o estructuras para guardar los nombres y las notas de todos los alumnos al mismo tiempo.

Agregar una función extra que calcule el promedio general del curso.

7. Conclusiones

Se cumplió con el objetivo de la práctica al aplicar la programación modular para resolver el cálculo de notas finales, logrando un código organizado donde cada función (ACD, APE, AA y ES) procesa su propio componente evaluativo y retorna los resultados ponderados necesarios para determinar el estado final del estudiante.

8. Recomendaciones

Se recomienda integrar estructuras de control como bucles y arreglos en futuras versiones del programa para permitir el ingreso masivo de datos, lo que facilita el manejo de notas de varios estudiantes de forma simultánea y eficiente

9. Bibliografía / Referencias

- [1] M. Goin, Caminando junto al Lenguaje C. Río Negro, Argentina: Editorial UNRN, 2022. [Online].
Available:
https://editorial.unrn.edu.ar/index.php/catalogo/346/view_bl/62/lecturas-de-catedra/26/caminando-junto-al-lenguaje-c?tab=getmybooksTab&is_show_data=1



Universidad
Nacional
de Loja

[2] J. E. Guerra Salazar, M. V. Ramos Valencia, and G. E. Vallejo Vallejo, Programando en C desde la práctica: problemas resueltos. Puerto Madero: Puerto Madero Editorial, 2023. [Online]. Available: <https://dialnet.unirioja.es/servlet/libro?codigo=933288>